

Helix Thready — OpenDesign Tooling Capability Check

Table of contents

1. Scope & method

2. The verified DESIGN.md brand-contract format

3. Local runtime status (checked 2026-07-22)

4. What runs headless now vs. what is operator-interactive

5. Figma import flow (exact, from figma-plugin/ source)

6. Honest limits

7. Open items

Rev	Date	Author	Change
1	2026-07-22	swarm (design · opendesign)	Initial honest capability check: verified DESIGN.md format, live daemon status, od CLI invocability, headless-vs-interactive matrix, exact Figma import flow, limits + open items
2	2026-07-22	swarm (design · opendesign · verify)	Second-pass verification: added the contributor authoring guide (docs/design-systems.md) findings — the second canonical heading family, the number-prefix-only section parser, Lens A/ B review checks (font-labels block, :root {} , [data-theme="dark"] , targeted prefers-reduced-motion) — and the measured corpus split (85 vs 61 of 150). Re-probed daemon health + CLI live (library list --json → empty library OK); confirmed local export toolchain paths

Table of contents

- 1. Scope & method

- 2. The verified DESIGN.md brand-contract format
- 3. Local runtime status (checked 2026-07-22)
- 4. What runs headless now vs. what is operator-interactive
- 5. Figma import flow (exact, from figma-plugin/ source)
- 6. Honest limits
- 7. Open items

1. Scope & method

Every claim below was checked directly against the local OpenDesign checkout at `/home/milos/Factory/projects/tools_and_research/.opendesign-src/open-design` and the live processes on this machine on 2026-07-22. `[VERIFIED]` = observed in source/runtime; `[ASSUMED]` = inference, labeled as such. Nothing is claimed to “work” without a runtime probe.

2. The verified DESIGN.md brand-contract format

There **is** a canonical format, and it is documented + exemplified in the OpenDesign source `[VERIFIED]`:

- Named in the product spec:** `docs/spec.md` §2 (bet #4): design systems are authored as “*DESIGN.md files following the [awesome-claude-design] 9-section schema*”; §4/§4: the `design-system-skill` “*produces a DESIGN.md following the 9-section format*”.
- Defined in `design-systems/README.md`:**
 - **Legacy file shape:** first `# H1` is the picker title; “*the line immediately after the H1 is parsed for > Category: <name>*” plus a one-line description blockquote.
 - **Project shape (v1):** `design-systems/<slug>/` with fixed names — `manifest.json` (schema `od-design-system-project/v1`), `DESIGN.md` (canonical prose), `tokens.css` (canonical compiled custom properties), optional `USAGE.md` / `components.html` / `design-tokens.json` / `tailwind-v4.css` / `assets/` / `fonts/` / `preview/` / `source/`. `DESIGN.md`-only folders remain valid (“legacy”).
- The 9 sections — two verified families coexist in the corpus.** The schema parser extracts headings with the **number prefix only** (`## [0-9].`); the text after the number is free `[VERIFIED — docs/design-systems.md §1: “matches the section number prefix, not the full text”]`. Measured across the 150 bundled `design-systems/*/DESIGN.md`:
 - **Family A (awesome-design-md import style, 85 files at `## 2.`, e.g. `design-systems/claude/DESIGN.md`;** unnumbered in the hand-authored `design-systems/default/DESIGN.md`): 1. Visual Theme & Atmosphere · 2. Color Palette & Roles · 3. Typography Rules · 4. Component Stylings · 5. Layout Principles · 6. Depth & Elevation · 7. Do’s and Don’ts · 8. Responsive Behavior · 9. Agent Prompt Guide.

- **Family B (contributor authoring-guide canon, 61 files**, e.g. `design-systems/mission-control/DESIGN.md` — the guide’s named reference): 1. Visual Theme & Atmosphere · 2. Color · 3. Typography · 4. Spacing · 5. Layout & Composition ·

1. Components · 7. Motion & Interaction · 8. Voice & Brand · 9. Anti-patterns `[VERIFIED — docs/design-systems.md §1 "The 9-Section Schema"]`. Both parse identically (number-prefix match). `./DESIGN.md` uses Family A headings and folds Family B’s distinct content (spacing, motion, voice & brand, anti-patterns) into the matching numbered slots. 3b. **Review framework (Lens A / Lens B)** `[VERIFIED — docs/design-systems.md §2]`. New design systems are reviewed against: **Lens A (blocking)** — all 9 numbered headings in order; real hex codes; CSS variables wrapped in `:root {}` (never bare); a “**Font labels for catalog extraction**” block in the Typography section (`Display:` / `Body:` / `Mono:` lines — the daemon’s parser regexes read these to populate the catalog); dark tokens via the `[data-theme="dark"]` override pattern (not duplicate blocks); `prefers-reduced-motion` targeting specific elements (never a global `*`); `:focus-visible` on every interactive component. **Lens B (advisory)** — ≥4 type tiers; real CSS in the Components section; specific, bounded anti-patterns; dark mode a genuine override; named prior art. WCAG AA 4.5:1 verification against the **paired** background is mandatory. `DESIGN.md` rev 2 satisfies these in-file (font-labels block, `:root` + `[data-theme="dark"]` blocks, verbatim `.ds-btn` CSS with a targeted reduced-motion gate).

4. **The token contract for `tokens.css`** : `packages/contracts/src/design-systems/token-schema.ts` (re-exported by `design-systems/_schema/tokens.schema.ts` ; A2 fallbacks mirrored in `_schema/default.ts.css`) defines four layers — **A1-identity** (required, brand-defining), **A1-structure** (required, structural), **A2** (required with a schema fallback), **B-slot** (optional tier, aliased via `var()`), plus per-brand **C-extensions** allowlisted in `BRAND_EXTENSIONS` . Every brand `tokens.css` must declare every A1 + A2 + B-slot token in one pasteable `:root` block.

5. **Single-mode caveat** `[VERIFIED]` : no bundled `tokens.css` contains `prefers-color-scheme` or `data-theme` — bundled systems are single-mode; dark palettes exist only as prose in some `DESIGN.md` files (e.g. `claude`, `linear-app`). Thready’s `tokens.css` adds dark blocks per the Thready ground truth (`./theming.md` §2 three sanctioned mechanisms) — a documented, additive extension: the light `:root` block alone still satisfies the schema.

`./DESIGN.md` follows the 9-section schema exactly (numbered variant), with the `> Category:` blockquote immediately after the H1 as the picker parser requires.

3. Local runtime status (checked 2026-07-22)

Check	Result	Evidence
Source checkout	present	<code>/home/milos/Factory/projects/tools_and_research/.opendesign-src/open-design</code> (git repo, HEAD of 2026-07-08 clone)
Version	0.14.1	<code>package.json</code> / daemon health. <code>Δ ../design-system.md</code> \$2 cites release 0.13.0 “at time of writing” — the local checkout has moved on; re-verify token-schema drift when integrating
Daemon live	running now	<code>GET http://127.0.0.1:7456/api/health</code> → <code>{ "ok": true, "version": "0.14.1" }</code> ; listener pid 2951699, loopback-only ; 442 bundled plugins registered (<code>daemon7456.log</code>)
<code>od</code> on PATH	not installed	<code>which od-cli od opendesign</code> → none. <code>Δ</code> Name collision: <code>/usr/bin/od</code> is GNU coreutils <i>octal dump</i> 9.4 — NOT OpenDesign. Never assume <code>od</code> on PATH is the design daemon
<code>od</code> CLI runnable	from the repo	<code>package.json</code> <code>"bin": { "od": "./apps/daemon/bin/od.mjs" }</code> ; probe <code>node ./apps/daemon/bin/od.mjs --help</code> → exit 0, full usage printed
<code>od</code> CLI ↔ daemon round-trip	live	<code>node apps/daemon/bin/od.mjs library list --json</code> → <code>{ "assets": [] }</code> (auto-discovered the <code>:7456</code> daemon; empty library is the expected fresh state — <code>.od/library/</code> and <code>.od/design-systems/</code> are empty dirs)
Toolchain		node v24.18.0, pnpm 10.33.2, <code>node_modules/</code> installed (2026-07-08)
Bring-up script	exists	<code>../opendesign-src/bringup.sh</code> — bounded orchestration: wait-for-clone → <code>pnpm install</code> (≤600s) → detached <code>pnpm tools-dev run web</code> → health-poll <code>:7456</code> (≤180s); emits a greppable <code>RESULT:</code> line
Historical logs		<code>daemon.log</code> : a 2026-07-08 <code>tools-dev</code> run (Web <code>:40425</code> , Daemon <code>:46591</code>) later stopped; <code>daemon7456.log</code> : the standalone <code>:7456</code> daemon start. Recorded pids 635028/639697 are dead — the current listener (2951699) is a later start

Check	Result	Evidence
Brands engine	exists	<code>apps/daemon/src/brands/engine/export.ts</code> exports <code>tokensToJson</code> , <code>tokensToCssVars</code> , <code>tokensToThemeJson</code> (grep-verified) — the token export pipeline <code>../design-system.md</code> §2 documents

4. What runs headless now vs. what is operator-interactive

Headless, today, on this machine `[VERIFIED — daemon live + CLI --help]`:

- **Daemon API** on `http://127.0.0.1:7456` (health, artifact store, design-system resolver, plugin registry).
- **od CLI** via `node ./apps/daemon/bin/od.mjs ...` (or `pnpm exec od` in-repo):
 - `od artifacts create --name <path> --input <file>` — create project artifacts;
 - `od tools design-systems read --path <manifest-declared-path>` — read the active design-system pull-layer files (i.e. this `DESIGN.md` / `tokens.css` shape);
 - `od library <list|get|search|import|apply|edit-as-page|figma|sync|pair>` — asset library, incl. `od library figma <assetId> --out page.od-figma.json` (export the Figma capture IR);
 - `od automation <...>` — drive Automations headlessly (“so an external agent ... can schedule, trigger, or harvest results ... without opening the web UI” — its own help text);
 - `od plugin <...>`, `od memory tree <...>`, `od mcp live-artifacts` (MCP server), `od research search` (needs a Tavily key — not probed).
- **Token derivation/export** via the brands engine (`tokensToJson` / `tokensToCssVars` / `tokensToThemeJson`) inside the daemon.

Needs a browser (local web UI, not headless) `[VERIFIED — spec.md §5]`:

- The Next.js web app (chat, artifact tree, iframe preview, comment mode, exports) — served locally by `pnpm tools-dev run web`; it is a UI on top of the same daemon.
- The **OD Clipper** browser extension (page capture) — operator-interactive by nature.

Needs a desktop application (operator-interactive or unprobed here):

- **Figma import** — requires the **Figma desktop app** (§5); development plugins do not run in the browser. This is the one third-party desktop dependency in the flow.
- `od export <file> --project <id> --format <pdf|image|pptx>` — programmatic artifact export exists in the CLI, but “*rasterization uses the desktop runtime’s bundled Chromium, so a desktop/packaged runtime must be reachable; otherwise the command reports that the renderer is unavailable*” `[VERIFIED — apps/daemon/src/cli.ts printExportHelp, verbatim]`. The OpenDesign desktop

shell is **not running** here, so this leg is unprobed — treat `od export` as **needs-the-desktop-app** for now.

- ⚠ Doc-drift note: `docs/spec.md` §6 still lists “No Electron, no Tauri” as a non-goal, but the 0.14.1 checkout **does** ship `apps/desktop` (`@open-design/desktop` 0.14.1, an Electron shell — `electron` in `package.json > onlyBuiltDependencies`) and `apps/packaged` `[VERIFIED — both inspected]` . Trust the code, not the stale spec line.
- Agent-driven generation additionally requires a code-agent CLI (Claude Code, Codex, ...) or a BYOK Anthropic key — present-agent dependent, not probed here `[ASSUMED — spec.md §2]` .

Local export toolchain (the working alternative, verified on this machine):

Thready screen artifacts are self-contained HTML inlining `tokens.css` , so the export pipeline that already produced the sibling `*.html` / `*.pdf` docs works without any OpenDesign desktop runtime

`[VERIFIED — command -v probes, 2026-07-22]` :

Tool	Path	Role
Chromium (headless)	<code>/usr/bin/chromium</code> (+ <code>chromium-browser</code>)	HTML → PNG/PDF screenshots (<code>--headless --screenshot/--print-to-pdf</code>)
WeasyPrint	<code>/home/milos/Factory/software/weasyprint/bin/weasyprint</code>	HTML/CSS → paginated PDF
Pandoc	<code>/home/milos/Factory/software/pandoc/bin/pandoc</code>	md → HTML (Docs Chain §11.4.65)
Mermaid CLI	<code>mmdc</code> (node 24 global)	<code>diagrams/*.mmd</code> → SVG/PNG

5. Figma import flow (exact, from figma-plugin/ source)

`figma-plugin/` is a standalone, no-build Figma **development plugin** (`manifest.json` : name “OD Figma Import”, `main: code.js` , `ui: ui.html` , `networkAccess.allowedDomains: [ "none" ]`) `[VERIFIED — figma-plugin/{README.md,manifest.json,IR.md}]` . It **rebuilds** an OD capture into editable Figma layers via the Plugin API — there is **no cloud push**: a native `.fig` cannot be produced outside Figma, and dragging an `.od-figma.json` into Figma shows “Unsupported file format” (expected, per the plugin README).

The exact three-step flow (all steps operator-interactive except step 1’s CLI variant):

1. **Produce a capture** (`.od-figma.json`) — any of:
 - OD Clipper popup → *Download Figma (.json)* (captures the current page);
 - OD Library → open an `html` asset → *Download Figma JSON*;
 - headless: `od library figma <assetId> --out page.od-figma.json` .

2. **Install the plugin once per machine** — requires the Figma **desktop** app (dev plugins do not run in the browser): open any file → **Plugins** → **Development** → **Import plugin from manifest...** → select `figma-plugin/manifest.json` from the checkout.
3. **Import** — **Plugins** → **Development** → **OD Figma Import** → drop/choose the `.od-figma.json` (or paste the JSON) → **Import**. The capture is rebuilt as a single frame named after the page, selected and zoomed.

Fidelity caveats `[VERIFIED — README.md "Fidelity notes" + IR.md]`: geometry is the captured live-page geometry; unavailable fonts fall back to **Inter Regular**; non-PNG/JPEG images (SVG/WebP/GIF/AVIF) are re-encoded to PNG in the plugin UI (SVGs rasterized, not editable vectors); complex CSS (multi-layer gradients, blend modes, transforms, pseudo-elements) is simplified; inset shadows skipped (IR v1). The capture IR contract is `figma-plugin/IR.md` (kept in sync with `clipper/capture.js`).

Thready relevance: design artifacts generated against `./DESIGN.md` + `./tokens.css` can be captured and hand-imported into Figma by an operator for design review. This is a *review/handoff* bridge, not a design-system sync — PenPot/Lottie and any true Figma round-trip remain non-native `[OPEN: THREADY-DES-02 — ../design-system.md $2]`.

6. Honest limits

- **No Figma token exists on this machine, and none is needed by this tooling** — the plugin's manifest pins `networkAccess.allowedDomains: ["none"]`; it cannot reach the Figma cloud API at all `[VERIFIED]`. Any “pushed to Figma cloud” claim from this environment would be a fabrication — do not make it. The only path into Figma is the operator-run desktop-app import above.
- The daemon is **loopback-only** here (`127.0.0.1`) — nothing is exposed off-machine `[VERIFIED — ss output + bringup.sh OD_BIND_HOST]`.
- End-to-end **generation** (brief → artifact) additionally depends on a detected code-agent CLI or a BYOK key; that leg was **not probed** in this pass `[ASSUMED — spec.md $2/$5]`.
- Version drift: local checkout is **0.14.1** while `../design-system.md` referenced 0.13.0; the token schema verified here is the 0.14.1 one.

7. Open items

- `[OPEN: THREADY-DES-0D-01]` If/when the Thready folder is vendored into `open-design/design-systems/thready/`, register the C-extension tokens (`--brand`, `--brand-2`, `--brand-ink`, `--border-strong`, `--ds-heart`, `--theme-id`) in `BRAND_EXTENSIONS["thready"]` and author the v1 `manifest.json` (`od-design-system-project/v1`) — legacy `DESIGN.md`-only shape is valid meanwhile.
- `[OPEN: THREADY-DES-0D-02]` Dark-mode blocks in `tokens.css` exceed the (single-mode) bundled precedent; confirm the 0.14.1 guard (`pnpm guard`) accepts the extra selectors, or split light/dark into the derive pipeline (`tokensToCssVars(tokens, selector)` already takes a selector argument).

- `[OPEN: THREADY-DES-OD-03]` Link this `opendesign/` folder from `../index.md` (this task wrote only into `opendesign/` by mandate).
 - `[OPEN: THREADY-DES-02]` (carried) PenPot + Lottie exports are not native OpenDesign targets; Figma is import-only via the operator flow in §5.
 - Re-verify the 0.13.0 → 0.14.1 schema drift against `../design-system.md` §2 at integration.
-

Made with love ♥ by Helix Development.